

Self - Directed Project:

Infinity Mirror



Amaiam ul Haque

Table of Contents

1) Scenario	3
a) Summary	3
b) Operating Instructions	3
2) Problems & Possibilities	5
3) Investigation & Ideas	6
a) Research Findings	9
b) Appendix	10
4) Construction	12
a) Parts List	12
b) Rough Sketches	13
c) Schematic View	16
d) Construction Chart	17
e) Arduino Programs	20
f) Activity Log	39
g) Progress Pictures	40
5) Evaluation.....	48
a) Self Assessment	49
b) Rubric	52
6) Bibliography	55

Scenario

This does not relate to any real-world problem. It is simply something made for pure recreational and entertainment purposes.

Summary

The infinity mirror is mainly for decorative purposes, however it serves as a regular mirror while powered off. However, the entire story changes when it is turned on. The smooth vivid transition that greets the user as soon as they turn it on, along with the various settings that allow the user to change it to their liking. It is designed purely to suit user preferences and convenience. Well the user in this case is me, since this is made and tweaked to my liking. As a result, the final product is a success if I am satisfied with it. When I first started looking into this, initially it seemed much more difficult and I did not think that the colour transitions would be *that* eye-catching. But after finally starting the project and getting into the colour transitions, I have never been so immersed with just lights. When I first succeeded in getting the transitions to finally work, I stared at them to the point where my eyes began to hurt. Therefore, I would find this to be a major success.

Operating Instructions

IR Remote Button Diagram:



Instructions:

**Note: IR Remote diagram shows the various buttons and what they are each for.*

- 1) Ensure that the power supply is charged.
- 2) Turn on the infinity mirror with the use of the remote control.
 - a) If it does not turn on, check if the power supply is off by taking a look at the back panel. The power supply should have a number displaying how much it is charged out of 100%.
 - b) If it does not display a number, simply press the only button found on the side.
 - c) If that does not work, it may just need to charge.
- 3) The default setting will automatically start on start up.
- 4) To swap through the various colour transitions, click the transition button until you are satisfied with one.
- 5) If at any given time, there is a colour that is displayed and you would like it to stay as it is, click the pause button.
 - a) To unpause, simply click the exact same button again.
- 6) To increase or decrease the brightness, click increase / decrease brightness in the respective order.
- 7) To increase or decrease the speed, click increase / decrease speed in the respective order.
- 8) In order to choose purple, click the purple button.
- 9) In order to choose white, click the purple white.
- 10) In order to choose warm white, click the warm white button.
- 11) In order to choose icy white, click the icy white button.
- 12) In order to turn it off, simply just click the power button again.

Core Components:

The IR remote and receiver uses interrupts in order to allow for the user's input to instantly start without waiting for the current section of code to finish. It also allows for the most convenient user interface that adjusts to their liking.

The individually addressable RGB LED strip allows for each and every single LED to be programmed separately instead of only being able to change the entire strip to a single colour. The desired user input to be outputted to showcase whatever the user wants to see. It is solely for visual purposes.

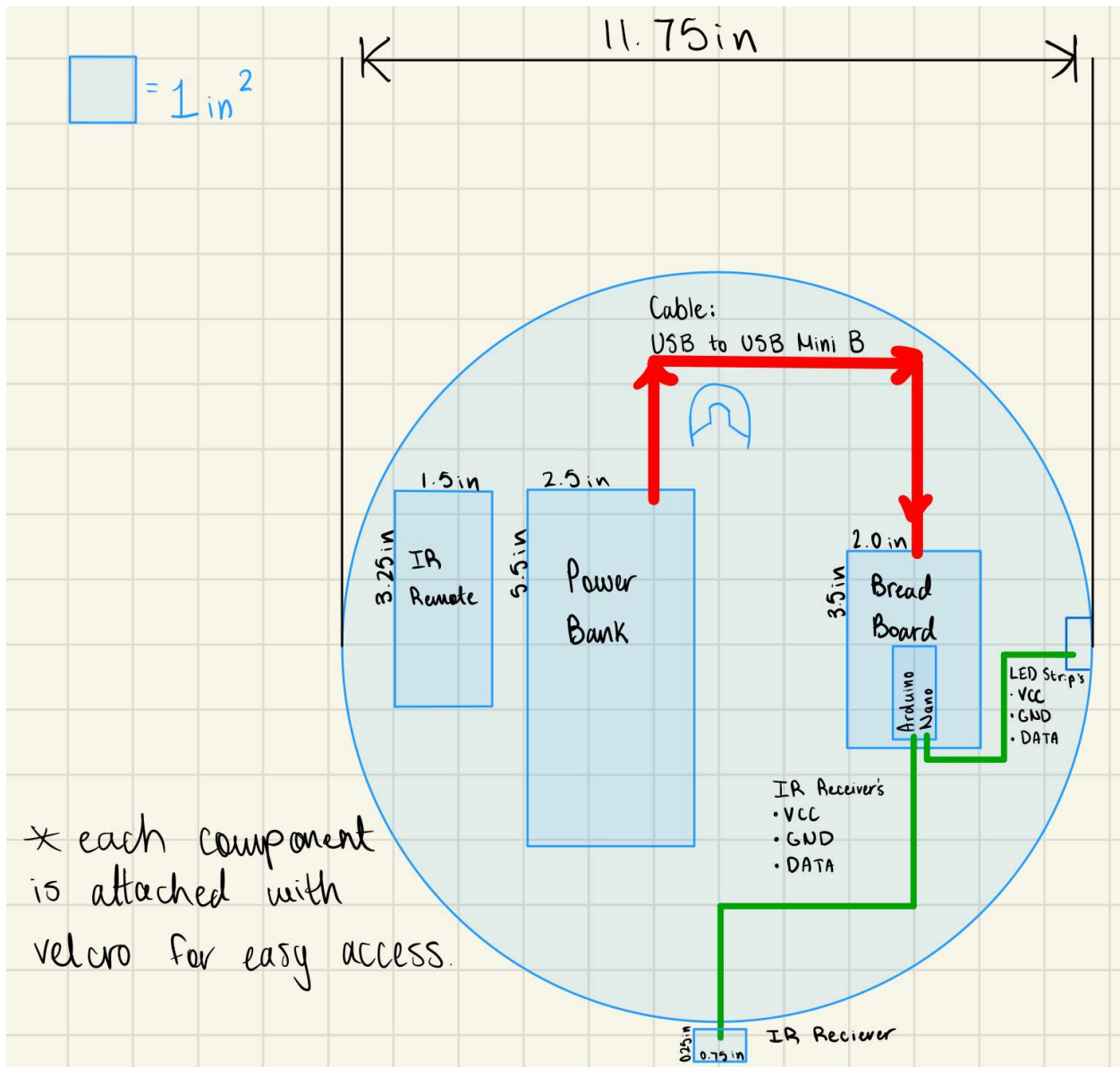
Problems & Possibilities

There were only a few, yet crucial limitations to this project.

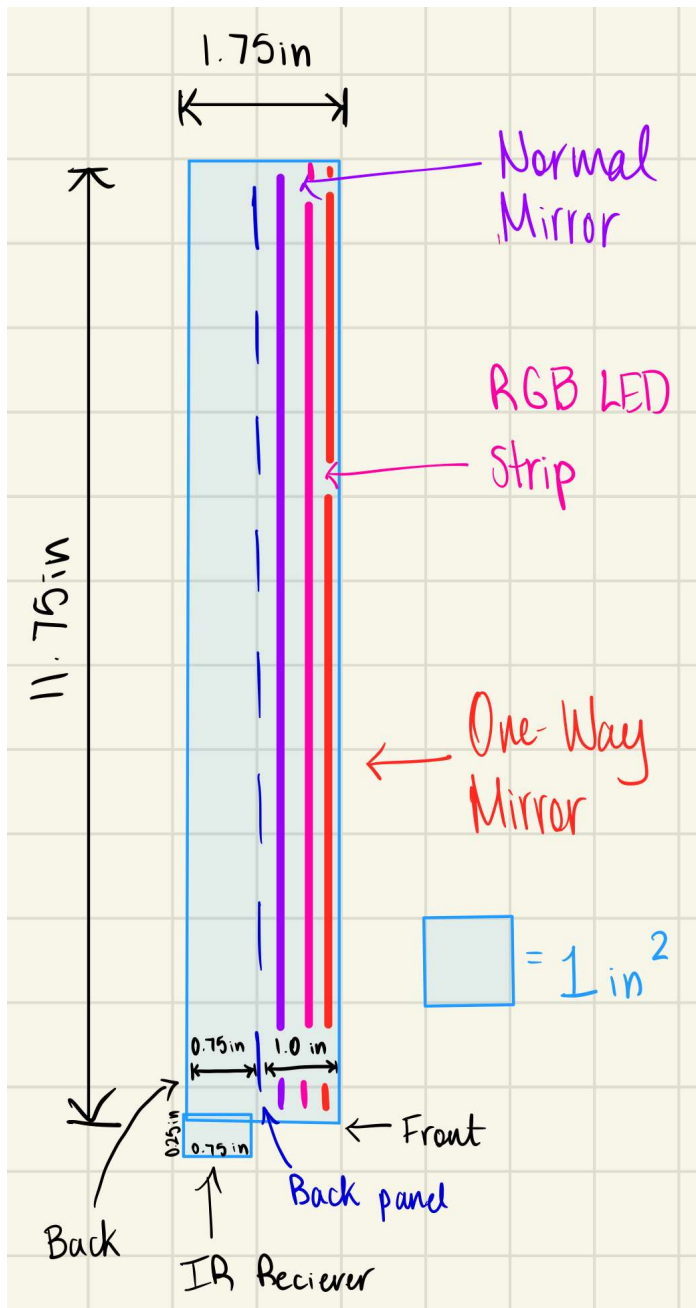
One of the major limitations would be time constraints. If I had more time, I would have liked to program a few more features. I had originally planned to add in a setting that would allow users to set their own colour transition range. For example they would be able to choose a starting and ending colour and the LEDs would only display that range of colours in whatever transition they would have liked, just as I have my own custom transitions of purples, since it is my favourite colour. Another feature would be adding in a default startup. With this users would be able to set up their favourite transitions along with the colours that they like, without having to go through the hassle of setting it up every single time they turn on the LEDs. The last feature that I wanted to add would be an additional colour transition. It would resemble a reloading symbol, starting off at max brightness while leaving a fading trail following it.

Another limitation would be the resources department. When it came to the time where I had to stick the ordinary mirror to the back panel, if it had been left upside down for a prolonged period of time, it would slowly come off. Fortunately, it did not break, but it takes quite a bit of effort just trying to centre the mirror without opening it up again. I used double sided tape, I would use a stronger adhesive next time, such as gorilla glue.

Back View:

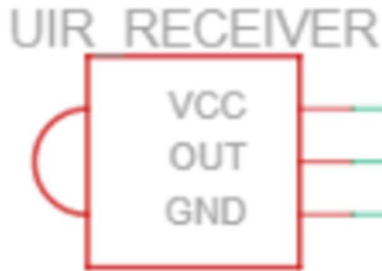


Side View:

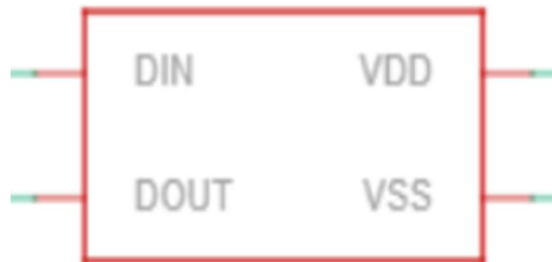


IR Receiver:

There is a pin for each VCC, which provides voltage to the component, GND, which is an exit for the current to return back to VCC and get power again, and then there is OUT. It is the data pin for the component, it receives incoming data.

RGB LED:

The pin for DIN (DATA IN) is responsible for all incoming data, while DOUT (DATA OUT) is responsible for all outgoing data. Usually to provide power there is VCC, which is for the entire circuit. But in this case there is VDD, which is the voltage for the chip and VSS is the voltage drainage system, in other words, GND.



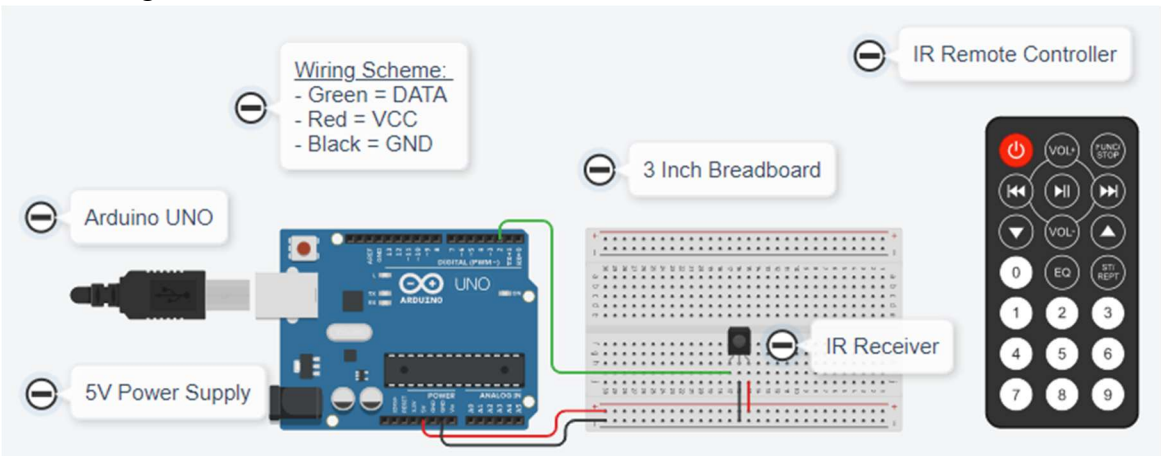
Research Findings

Through my journey through the internet, I have learned various concepts that are crucial for this project. The two main components work in ways that I have never seen before. The RGB LED strip uses a byte each to represent red, green, and blue. Since these are all the primary colours of light, with just these three, you can make all the colours of light, to be more specific 16'777'216 different colours. The website gave me an insight on the basic commands and uses of the FastLED library by Daniel Garcia. The second major component would be the IR receiver. It uses incoming infrared light signals from the remote to take in decoded information. The video I watched taught me about the basics of how to use the IR remote library by Ken Shiriff and how to decode my remote codes in order to customise it for my own use. Both of these tutorials gave me a gist of how everything worked and provided me with enough information that I code on my own from scratch.

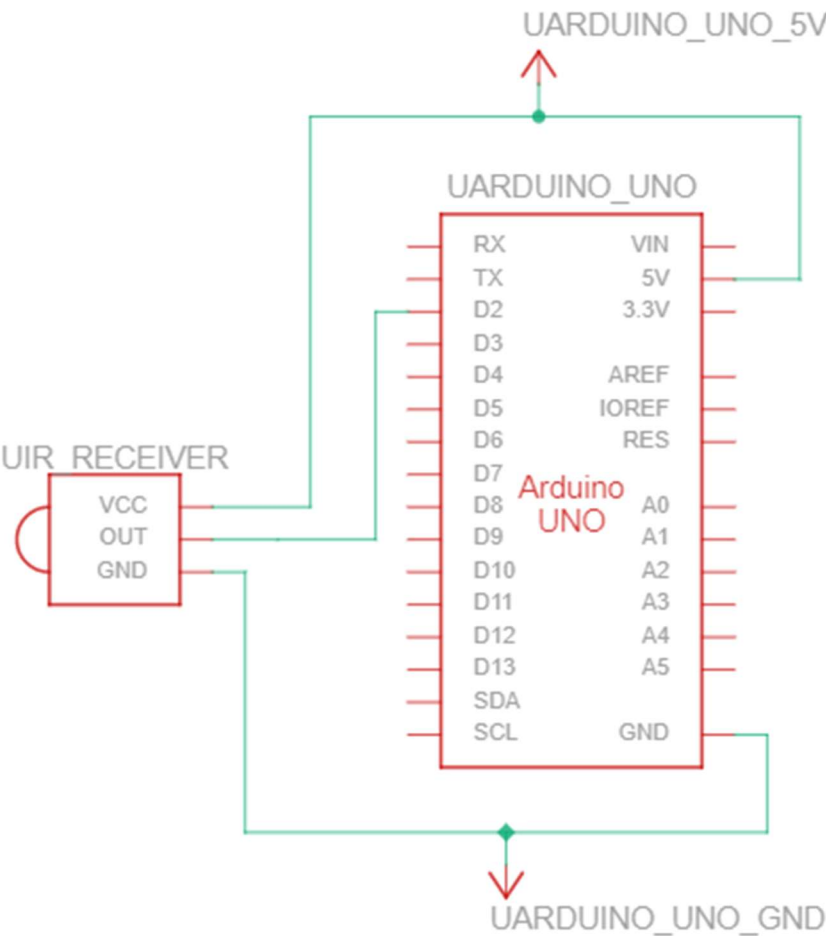
Appendix

IR Receiver Pinout Diagram:

Original Tinkercad Circuit:

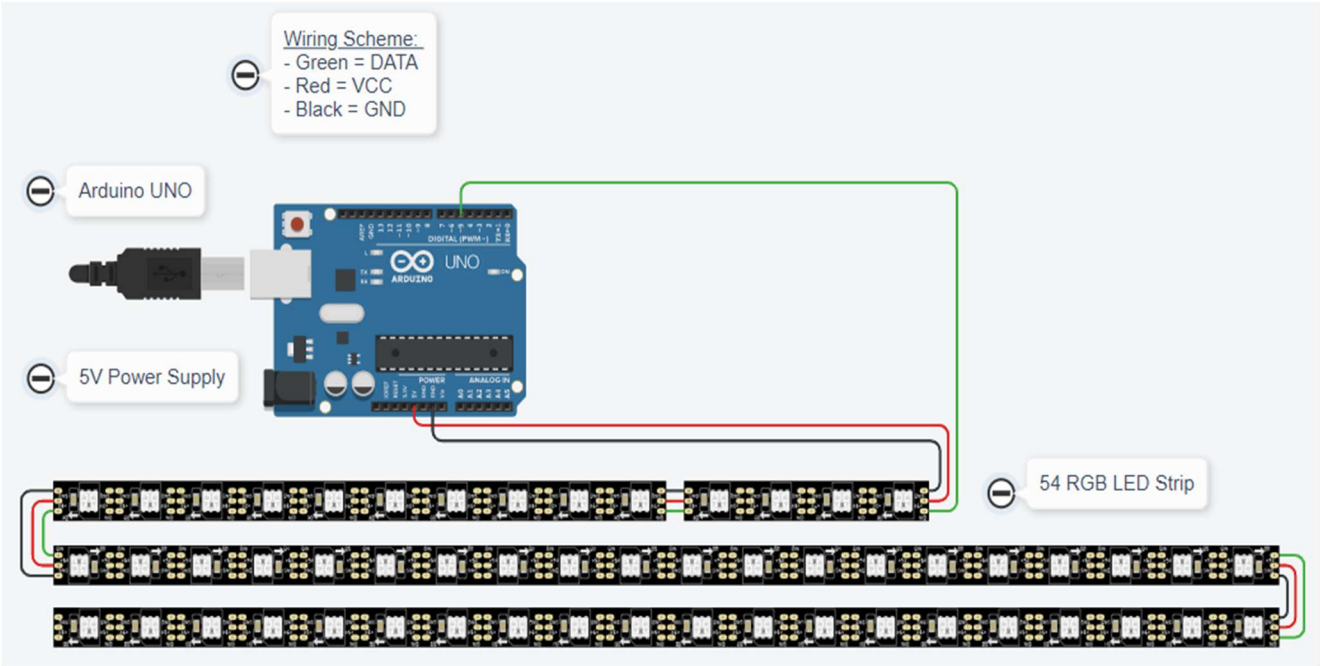


Schematic:

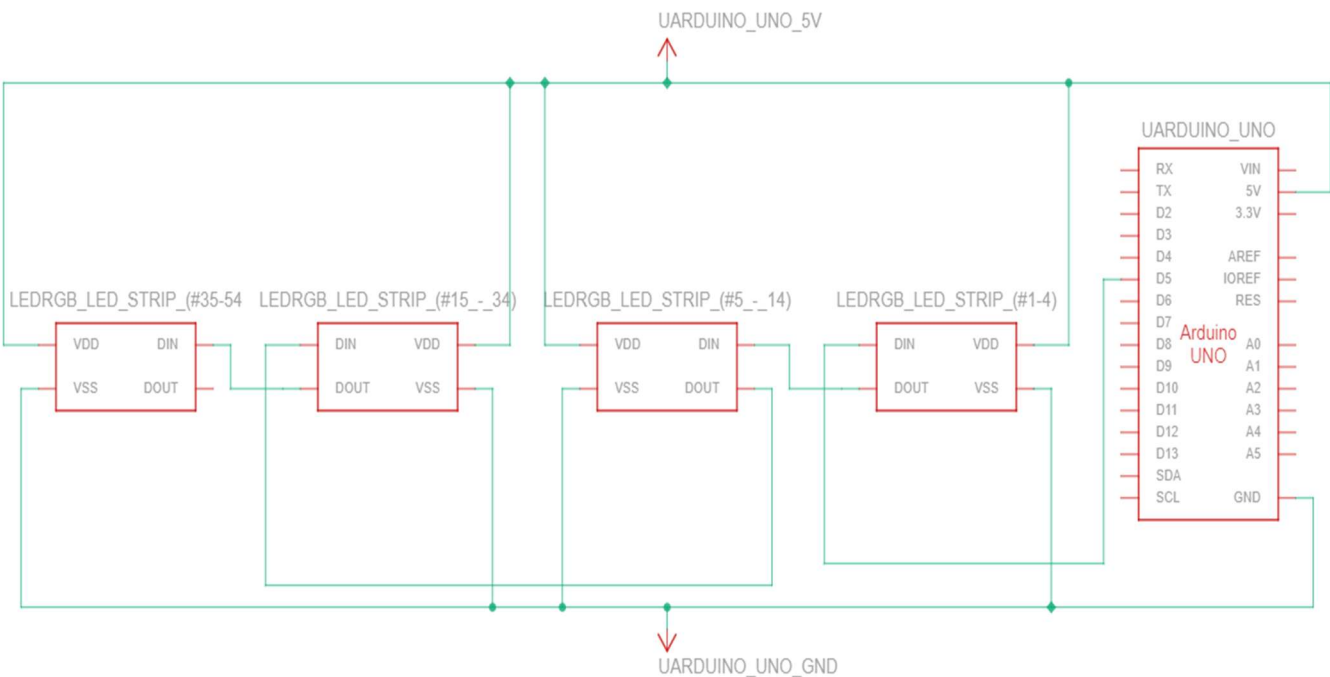


RGB LED Pinout Diagram:

Original Tinkercad Circuit:



Schematic:



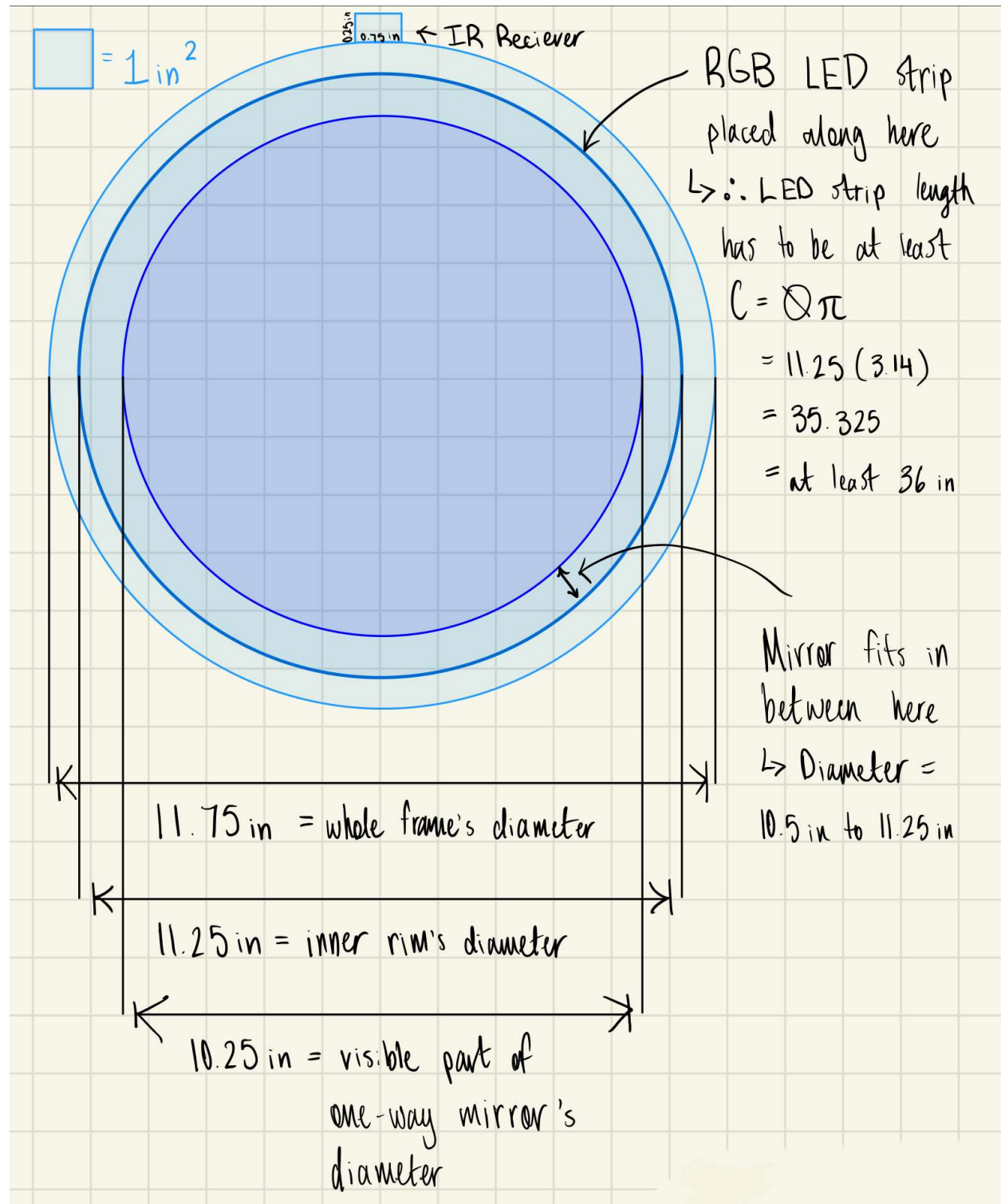
Construction

Parts List

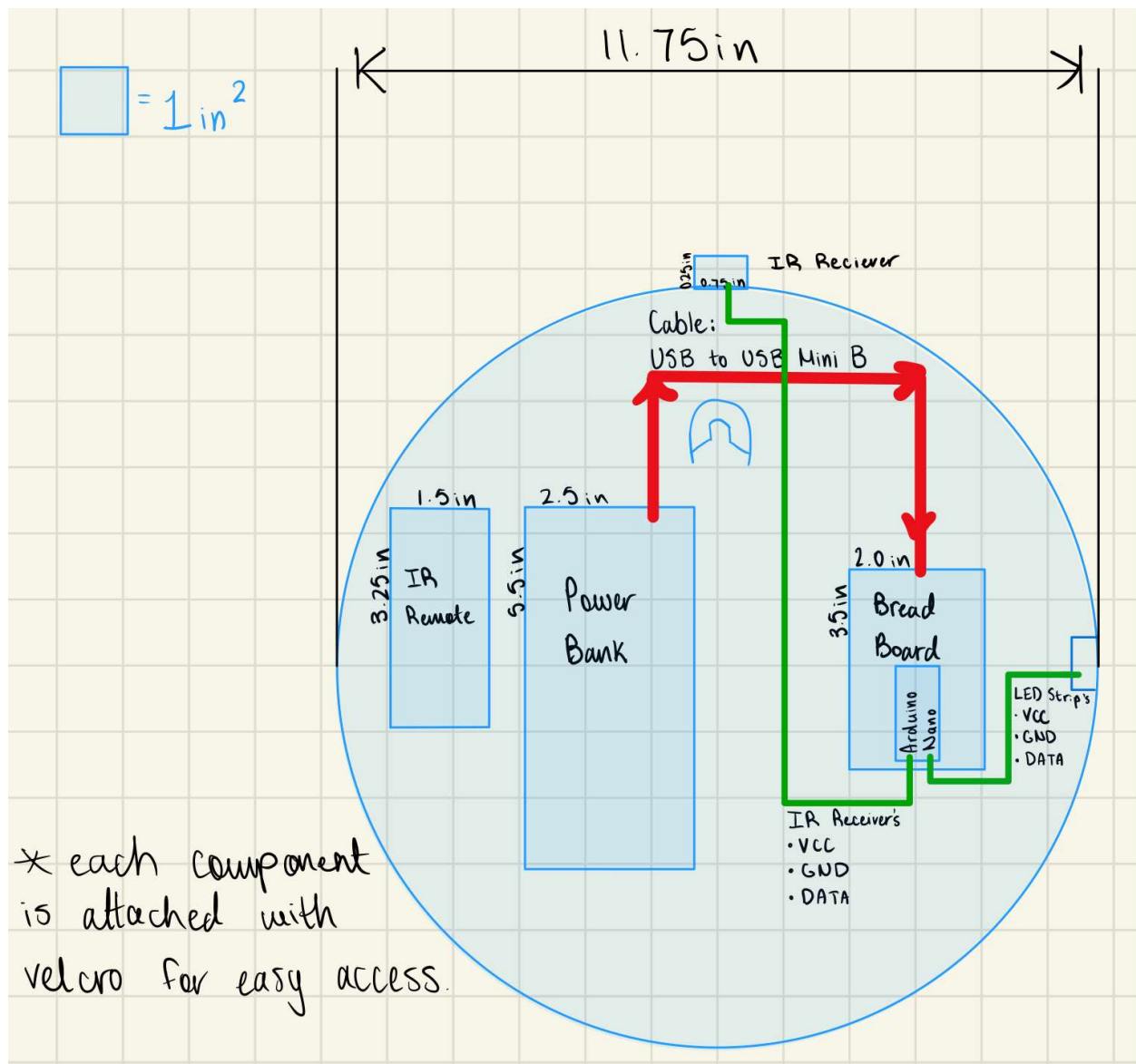
Quantity	Component Name
x 1	Arduino Nano
x 1	Male USB Type A to Male USB Micro
x 1	Breadboard 3 Inch
x 1	Individually Addressable RGB LED Strip 1m
x 1	Power Bank/Portable Charger "DC 5V 3A"
x 1	IR Remote Control with +8 buttons
x 1	IR Receiver Module
x 1	One Way Mirror Film
x 1	Clock Frame
x 1	Glass from Clock Frame
x 1	Circular Mirror (0.5 inches in diameter less than the clock frame)
x 1	Window Film Application Kit

Rough Sketches

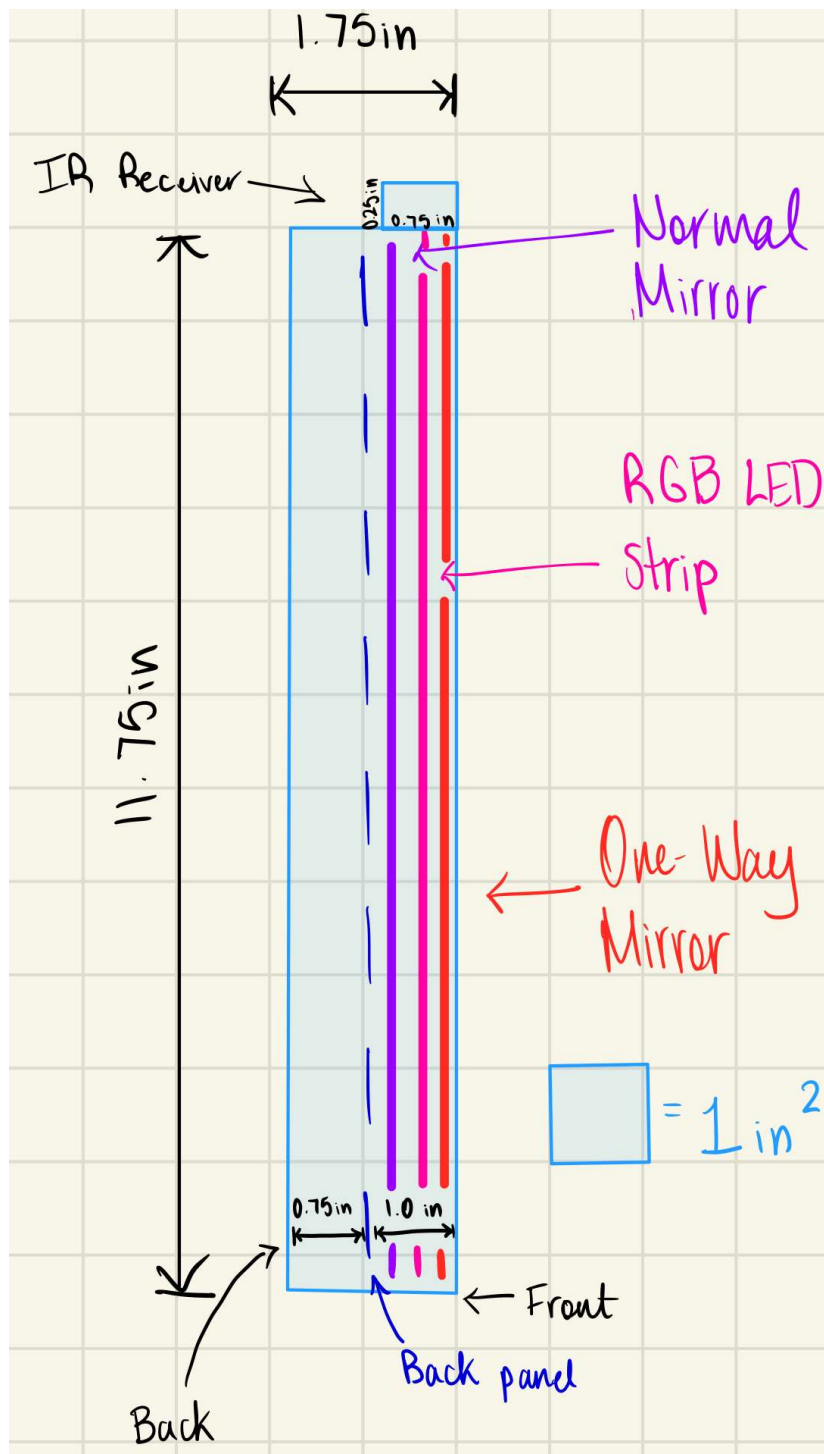
Front View:



Back View:

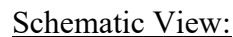


Side View:



In the end, the only difference was that the IR Receiver is now at the top front instead of being at the bottom back of the entire model in order to increase the range of the IR receiver.

Original Tinkercad View:



Construction Chart

**Note: For any doubts in wiring, refer to the wiring in the 'Schematic View' or 'Appendix' section.*

Step	Operation	Tips/Details	Equipment	Safety
1	Research	Used websites and videos to research how to code and wire both the RGB LED strip and IR receiver and remote control using their respective libraries and how to apply transparent mirror film.	-A device with an internet connection	N/A
2	Testing	Wired and tested the RGB LED strip ensuring that each of the colours (red, green, blue, white) work at max brightness (255) by uploading the code, <i>Sketch #3</i> in the <i>Arduino Program</i> section, into the Arduino.	-A device capable of uploading code onto the Arduino	Ensure to wire correctly by referring to the wiring in the <i>Schematic View</i> section, so no short circuits are caused, which would ultimately lead to the circuit catching on fire from the excess voltage overload.
3	Testing	Wired and tested out IR receiver and remote control and retrieved the specific codes by uploading <i>Sketch # 5</i> in the <i>Arduino Program</i> section to the Arduino.	-A device capable of uploading code onto the Arduino	Ensure to wire correctly by referring to the wiring in the <i>Schematic View</i> section, so no short circuits are caused, which would ultimately lead to the circuit catching on fire from the excess voltage overload. Additionally be careful of the IR receiver component, since it is spiky underneath as a result of the cut off parts.
4	Assembly	Measured, cut, and applied mirror film to glass by following the exact steps of the video found on this site, Window Application Process	-An exacto knife -A low lint	Always handle glass gently in an area free of clutter and always place it in an area where it is easily

			cloth -A solution (could be just water) in a spray bottle	visible to avoid any possible mishaps leading to broken sharp pieces of glass. Take caution when also using anything that involves a blade. When not in immediate use, put the cover on the blade to avoid any possible cuts.
5	Assembly	Temporarily align the LED strip along the inner rim of the back panel and measure how many LEDs can fit. Use tape to temporarily hold the strip down along the rim if it keeps moving if necessary. Then cut along the white thin line (there is one between each LED and there are three gold ovals on them).	-wire cutters	Take caution while using wire cutters as they could cause possible cuts. Ensure to enable the safety lock when not in immediate use.
6	Assembly	Use an exacto knife to make a hole right next to any spot along the inner rim of the back panel. Note that the hole needs to be big enough so that there is at least a bit of extra space for the widest part of the wires to go through (might have to consider size of solderless clip)	-Exacto knife	Take caution when also using anything that involves a blade. When not in immediate use, put the cover on the blade to avoid any possible cuts.
7	Assembly	Use a strong adhesive to stick the LED strip along the inner rim of the back panel. Begin with the starting of the LED strip from where the hole starts, since the wires have to go through it. Use tape if necessary to hold it temporarily while the adhesive dries if it needs to.	-A strong adhesive	Take caution when using a string adhesive. Especially gorilla glue. Wear gloves if necessary. If any adhesive sticks onto yourself, read safety steps on the product's label.
8	Assembly	Used black tape to tightly wrap around the LED wires, enough so that even if the wires rub against the edges of the hole, that they will not be damaged. Then, poked the wires through the hole of the back panel. Flipped the back	-Black roll of tape	Take caution of frayed or damaged wires. Use electrical tape if any problems arise. Be careful with glass and

		panel over and firmly taped to seal the hole. Then stuck the normal mirror to the back panel with a strong adhesive, while ensuring that it is centred.		mirrors as they can easily shatter. Always place them in a clearly visible area.
9	Assembly	Cleaned off the smudges on both the normal mirror and one way mirror with a glass cleaner. Placed the one way mirror on top of the front of the back panel while holding it from the sides ensuring to not smudge it.	-A low lint cloth	Take caution when handling mirrors as they can have sharp edges and can shatter easily.
10	Assembly	Gently placed and fitted the front part / screen onto the front of the back panel while ensuring the one way mirror is centred and remains unsmudged. Then slowly picked up and flipped the entire model over to firmly but gently press the front part / screen onto the back panel.	N/A	Be careful with glass and mirrors as they can easily shatter. Always place them in a clearly visible area.
11	Assembly	Temporarily placed all electrical components where they belong. Then picked up each component one by one and stuck a piece of velcro to the component and stuck the other piece directly onto the velcro piece, as velcro should stick to each other, and then placed the component on the exact spot that it was initially placed on.	N/A	Ensure to place all components in a clearly visible area neatly as to avoid any possible damage and frayed wires.
12	Coding	Wired and uploaded the final version of the code onto the Arduino using <i>Sketch #6</i> in the <i>Arduino Programs</i> section.	-A device capable of uploading code onto the Arduino	Ensure to place all components in a neat and clear area to avoid damaging and fraying wires.
13	Assembly	Finally, tidied up wires on the back panel either using tape, zip ties, or anything that can hold its shape.	-Tape / zip ties / etc...	N/A

Arduino Programs

Sketch 1: Original Code Found Online for LEDs

```
//link: https://www.makerguides.com/how-to-control-ws2812b-individually-addressable-leds-using-arduino/
#include <FastLED.h>

#define RGB_PIN          6           // LED DATA PIN
#define RGB_LED_NUM      10          // 10 LEDs [0...9]
#define BRIGHTNESS      200         // brightness range [0..255]
#define CHIP_SET         WS2812B    // types of RGB LEDs
#define COLOR_CODE       GRB         //sequence of colours in data stream

// Define the array of LEDs
CRGB LEDs[RGB_LED_NUM];

void setup() {
  Serial.begin(9600); //initialise monitor and baud rate
  Serial.println("WS2812B LEDs strip Initialize"); //program start
  FastLED.addLeds<CHIP_SET, RGB_PIN, COLOR_CODE>(LEDs, RGB_LED_NUM); //adds LEDs into a group to make it easier to use functions (e.g. setBrightness, clear, show, etc)
  FastLED.setBrightness(BRIGHTNESS); //sets the brightness of the LED strip to the value of BRIGHTNESS
  FastLED.setMaxPowerInVoltsAndMilliamps(5, 500); //sets the maximum power to ensure safe usage
  FastLED.clear(); //clears all local data in the LEDs, basically just wipes it clean from any previous data
  FastLED.show(); //used to update the Arduino about the status of the CRGB function
  //CRGB function responsible for the colours displayed to represent the red, green, and blue values which can range from a byte or 0 to 255
  //it does not show anything here though since all local data was just removed
  //this gives LEDs a fresh start
}

void loop() {
  Toggle_RED_LED(); //calls on the method 'Toggle_RED_LED'
}

void Toggle_RED_LED(void) {
  //for loop goes through each of the LEDs to make it turn red and then back to black with 1 second delay in between
  for (int i = 0; i < RGB_LED_NUM; i++)
    LEDs[i] = CRGB(255, 0, 0 ); //this is the rgb values needed to represent red
  FastLED.show();
  delay(1000);
}
```

```

for (int i = 0; i < RGB_LED_NUM; i++)
    LEDs[i] = CRGB(0, 0, 0);
FastLED.show();
delay(1000);
}

```

Sketch 2: Original Code Found Online for IR Receiver and Remote

```

#include <IRremote.h>      //includes the library needed for the IR remote
const int RECV_PIN = 2;    //declares the incoming DATA pin
IRrecv irrecv (RECV_PIN); //creates and names the receiver object
decode_results results;    //stores incoming code from 'irrecv'

void setup() {
    Serial.begin(9600); //initialises serial monitor and baud rate to 9600
    Serial.println("program start.");
    irrecv.enableIRIn(); //begins the receiver to receive incoming signals
}

void loop() {
    //ensure to press each button, and record the hexadecimal (the code that
    //is 6 letters and numbers long)
    //they will be needed later to code a certain task to each corresponding
    //button
    if (irrecv.decode(&results)) { //stores a code if a signal is received
        Serial.println(results.value, HEX); //prints out the code received in
        hexadecimal
        irrecv.resume(); //allows the receiver to continue to receive incoming
        signals
    }
}

```

Sketch 3: Testing Out All LEDs

```

#include <FastLED.h>          //includes the FastLED library
#define RGB_PIN              6      //declares LED strip DATA pin
#define RGB_LED_NUM          60     //declares how many LEDs are in the strip
#define BRIGHTNESS           255    //declares a brightness, which ranges from
0 to 255, equivalent to a byte
#define CHIP_SET              WS2812B //declares what type of RGB LEDs the strip
is
#define COLOR_CODE            GRB    //declares the order of the red, green,
and blue values
//in this case it is green, then red, then blue

// define the array of LEDs
CRGB LEDs[RGB_LED_NUM];

```

```

void setup() {
  Serial.begin(9600); //initialises serial monitor and baud rate to 9600
  Serial.println("program start.");

  FastLED.addLeds<CHIP_SET, RGB_PIN, COLOR_CODE>(LEDs, RGB_LED_NUM);
  //adds LEDs to make in easier to use functions such as setBrightness,
  clear, show, etc

  FastLED.setBrightness(BRIGHTNESS); //sets brightness according to the
  variable 'BRIGHTNESS'
  FastLED.setMaxPowerInVoltsAndMilliamps(5, 500); //sets maximum power for
  safe usage
  FastLED.clear(); //clears any local LED data
  FastLED.show(); //updates local RGB LED data
}

void loop() {
  //each function makes the enitre LED strip to display a single colour
  //red, green, and blue are the primary colours of light
  //with them you can make all the colours you want, specifiially upto
  16'777'216 colours
  //mix all three, and you get white
  //white is tested to ensure the LED strip can handle max power
  //purple is displayed only because it is my favourite colour

  AllRed();
  AllGreen();
  AllBlue();
  AllWhite();
  AllPurple();
}

//each method uses a for loop to set each LED in the array to the written
hexcode
//then clears the local data stored on it
//returning all the LEDs to black

void AllRed() {
  SetAllLEDs(255, 0, 0 );
  delay(3500);
  FastLED.clear();
  delay(1500);
}

void AllGreen() {
  SetAllLEDs(0, 255, 0 );
  delay(3500);
  FastLED.clear();
  delay(1500);
}

```

```

void AllBlue() {
    SetAllLEDs(0, 0, 255 );
    delay(3500);
    FastLED.clear();
    delay(1500);
}

void AllWhite() {
    SetAllLEDs(255, 255, 255 );
    delay(3500);
    FastLED.clear();
    delay(1500);
}

void AllPurple() {
    SetAllLEDs(255, 0, 255 );
    delay(3500);
    FastLED.clear();
    delay(1500);
}

void SetAllLEDs (byte r, byte g, byte b) {
    //sets all LEDS to a single colour using a for loop to go through each LED
    //only needs the parameters of each rgb value
    for (int i = 0; i < LED_NUM; i++)
        LEDs[i] = CRGB(r, g, b);
    FastLED.show();
}

```

Sketch 4 : IR Remote Switch Cases

```

#include <IRremote.h>      //includes the library needed for the IR remote
const int RECV_PIN = 2;   //declares the incoming DATA pin
IRrecv irrecv (RECV_PIN); //creates and names the receiver object
decode_results results;    //stores incoming code from 'irrecv'

void setup() {
    Serial.begin(9600); //initialises serial monitor and baud rate to 9600
    Serial.println("program start.");
    irrecv.enableIRIn(); //begins the receiver to receive incoming signals
}

void loop() {
    //ensure to press each button, and record the hexadecimal (the code that
    //is 6 letters and numbers long)
    //they will be needed later to code a certain task to each corresponding
    //button

    if (irrecv.decode(&results)) {          //stores a code if a signal is

```

```

received
    Serial.println(results.value, HEX); //prints out the code received in
hexadecimal
    irrecv.resume(); //allows the receiver to continue to
receive incoming signals

    switch (results.value) {
        //depending the results of the incoming signals, a certain task is
done
        //in this case it prints what button was pressed to the serial monitor
        case 0xFFA25D:
            Serial.println("CH- button has been pressed");
            break;
        case 0xFF629D:
            Serial.println("CH button has been pressed");
            break;
        case 0xFFE21D:
            Serial.println("CH+ button has been pressed");
            break;
        case 0xFF22DD:
            Serial.println("PREV button has been pressed");
            break;
        case 0xFF02FD:
            Serial.println("NEXT button has been pressed");
            break;
        case 0xFFC23D:
            Serial.println("PAUSE button has been pressed");
            break;
        case 0xFFE01F:
            Serial.println("VOL- button has been pressed");
            break;
        case 0xFFA857:
            Serial.println("VOL+ button has been pressed");
            break;
        case 0xFF906F:
            Serial.println("EQ button has been pressed");
            break;
        case 0xFF6897:
            Serial.println("0 button has been pressed");
            break;
        case 0xFF9867:
            Serial.println("+100 button has been pressed");
            break;
        case 0xFFB04F:
            Serial.println("+200 button has been pressed");
            break;
        case 0xFF30CF:
            Serial.println("1 button has been pressed");
            break;
        case 0xFF18E7:
            Serial.println("2 button has been pressed");
            break;
        case 0xFF7A85:
            Serial.println("3 button has been pressed");
            break;
    }

```



```

        case 0xFF10EF:
            Serial.println("4 button has been pressed");
            break;
        case 0xFF38C7:
            Serial.println("5 button has been pressed");
            break;
        case 0xFF5AA5:
            Serial.println("6 button has been pressed");
            break;
        case 0xFF42BD:
            Serial.println("7 button has been pressed");
            break;
        case 0xFF4AB5:
            Serial.println("8 button has been pressed");
            break;
        case 0xFF52AD:
            Serial.println("9 button has been pressed");
            break;
    }
}
}
}

```

Sketch 5: IR Remote Interrupt

```

#include <IRremote.h>    //includes the library needed for the IR remote
const int RECV_PIN = 2; //declares the incoming DATA pin
IRrecv irrecv (RECV_PIN); //creates and names the receiver object
decode_results results; //stores incoming code from 'irrecv'

void setup() {
    Serial.begin(9600); //initialises serial monitor and baud rate to 9600
    Serial.println("program start.");
    irrecv.enableIRIn(); //begins the receiver to receive incoming signals

    pinMode(RECV_PIN, INPUT_PULLUP); //declares the pin as an interrupt
    attachInterrupt(digitalPinToInterrupt(RECV_PIN), eee, CHANGE);
    //declares the pin number, the method name, and activation state in order
    for interrupt to run
    //there are three possible activation states
    //RISING activates when there is a rise in voltage
    //FALLING activates when there is a drop in voltage\
    //CHANGE activates when they is either a rise or drop in voltage
}

void loop() {
    Serial.println("on standby...");
    delay(2500);
}

```

```

void IRTranslate() {
    //this ensures that the interrupt is working if certain button outputs are
    //printed to the serial monitor
    //depending on what button is pressed, a certain task is carried out

    if (irrecv.decode(&results)) {           //stores a code if a signal is
received
        Serial.println(results.value, HEX); //prints out the code received in
hexadecimal
        irrecv.resume();                    //allows the receiver to contune to
receive incoming signals

        switch (results.value) {
            //depending the results of the incoming signals, a certain task is
done
            //in this case it prints what button was pressed to the serial monitor
            case 0xFFA25D:
                Serial.println("CH- button has been pressed");
                break;
            case 0xFF629D:
                Serial.println("CH button has been pressed");
                break;
            case 0xFFE21D:
                Serial.println("CH+ button has been pressed");
                break;
            case 0xFF22DD:
                Serial.println("PREV button has been pressed");
                break;
            case 0xFF02FD:
                Serial.println("NEXT button has been pressed");
                break;
            case 0xFFC23D:
                Serial.println("PAUSE button has been pressed");
                break;
            case 0xFFE01F:
                Serial.println("VOL- button has been pressed");
                break;
            case 0xFFA857:
                Serial.println("VOL+ button has been pressed");
                break;
            case 0xFF906F:
                Serial.println("EQ button has been pressed");
                break;
            case 0xFF6897:
                Serial.println("0 button has been pressed");
                break;
            case 0xFF9867:
                Serial.println("+100 button has been pressed");
                break;
            case 0xFFB04F:
                Serial.println("+200 button has been pressed");
                break;
            case 0xFF30CF:
                Serial.println("1 button has been pressed");

```

```

        break;
    case 0xFF18E7:
        Serial.println("2 button has been pressed");
        break;
    case 0xFF7A85:
        Serial.println("3 button has been pressed");
        break;
    case 0xFF10EF:
        Serial.println("4 button has been pressed");
        break;
    case 0xFF38C7:
        Serial.println("5 button has been pressed");
        break;
    case 0xFF5AA5:
        Serial.println("6 button has been pressed");
        break;
    case 0xFF42BD:
        Serial.println("7 button has been pressed");
        break;
    case 0xFF4AB5:
        Serial.println("8 button has been pressed");
        break;
    case 0xFF52AD:
        Serial.println("9 button has been pressed");
        break;
    }
}
}

```

Sketch 6: Final Version -

**Note - this is much neater in the .INO file.*

```

/*
  Amaiam ul Haque
  Last Updated: Jan 26, 2023

Description:*****
*****
  Let's-a-go! *insert super mario bros theme*
*/

/* Note:
 * Since this file has multiple tabs,
 * each of them run in a specific order.
 * It will always start off with the tab that the folder is named after,
 * and then the rest is determined by their alphabetical order.
 * Which is why each tab starts with a,b,c, so on and so forth,
 * and if there is a second letter following the first one,
 * this is meant to serve as some sort of a subsection.
 *
 * For example,
 * the transitions tabs start with the letter 'f',

```

```

* while each different type of transition have their own tab/subsection
(fa, fb, fc...)
*
* But in this case the only tabs that have to be in order are the
declarations, setup, and loop.
* Since all the other tabs are methods and will not run until called upon.
*/

#include <FastLED.h>           //the library for RGB LEDs
#define LED_PIN                5           //LED data pin
#define CHIP_SET                WS2812B    //type of RGB LED Strip
#define COLOUR_CODE            GRB         //the sequence of colours
#define LED_NUM                54         //# of LEDs in strip
CRGB LEDs[LED_NUM];           //array to hold all LEDs

#include <IRremote.h>          //library for the IR receiver
const int RECV_PIN = 2;       //incoming & outgoing signal pin
IRrecv irrecv (RECV_PIN);     //create and name receiver object
decode_results results;       //stores incoming code/results

int duration = 1;             //the delay between each colour/transition
byte brightness = 255;        //the desired brightness level of the LEDs
(0 = off, 1 = dimmest 255 = brightest)
boolean currentState = HIGH;   //tracks the current state of power (whether
the LEDs are on or off)
boolean paused = false;       //tracks whether or not the LEDs are
stopped/paused at a chosen colour
int transition = 0;           //tracks the current transition to switch
between each of them
byte crease = 15;             //the (in/de)crease / step for each colour
in between the fade transition
int currentLED = 0;           //keeps track of current LED in the array
for the spiral transition
int previousLEDs;             //keeps tracks of previous LEDs to update
their brightness
                                //currently this is not working how i want
it to
int previousResult;           //keeps track of the most recent input
received from the IR remote

void setup() {
  Serial.begin(9600);
  Serial.println("Let the show begin!!!"); //program has started

  //configuration of the RGB LED strip using the FastLED library
  FastLED.addLeds<CHIP_SET, LED_PIN, COLOUR_CODE>(LEDs, LED_NUM); //adds
LEDs so it is easier to use functions such as setBrightness, clear, and show
  FastLED.setMaxPowerInVoltsAndMilliamps(5, 3000); //sets max power to
ensure safe usage

```

```

FastLED.setBrightness(brightness); //sets the brightness of LEDs
FastLED.clear(); //clears and previous local data
FastLED.show(); //updates the local data and displays it

//configuration of the IR remote using the IRremote library
irrecv.enableIRIn(); //IR Receiver begins receiving process
pinMode(RECV_PIN, INPUT_PULLUP); //declares pin as input meant for an
interrupt
attachInterrupt(digitalPinToInterrupt(RECV_PIN), IRTranslate, RISING);
//declares the pin, method, & activation state to setup the interrupt
}

//void loop() { //this is what is supposed to run if the IRTranslate method
worked.
// Default();
//}

void loop() {
//since the IRTranslate method is not working, code will have to go
through them each manually
//everything below is just for easy copy paste

//sets brightness and duration to what a value that the IR remote could
have done and goes through each transition

brightness = 75;
FastLED.setBrightness(brightness);
duration = 10;
Default();

brightness = 175;
FastLED.setBrightness(brightness);
duration = 5;
Default();

brightness = 255;
FastLED.setBrightness(brightness);
duration = 1;
Default();

Fade(crease); //this one only has duration of one since it takes really
long

brightness = 75;
FastLED.setBrightness(brightness);
duration = 10;
for (int i = 0; i < 5; i++)
    Spiral();

brightness = 175;
FastLED.setBrightness(brightness);

```

```
duration = 5;
for (int i = 0; i < 7; i++)
    Spiral();

brightness = 255;
FastLED.setBrightness(brightness);
duration = 1;
for (int i = 0; i < 10; i++)
    Spiral();

brightness = 75;
FastLED.setBrightness(brightness);
duration = 10;
for (int i = 0; i < 5; i++)
    Reload();

brightness = 175;
FastLED.setBrightness(brightness);
duration = 5;
for (int i = 0; i < 7; i++)
    Reload();

brightness = 255;
FastLED.setBrightness(brightness);
duration = 1;
for (int i = 0; i < 10; i++)
    Reload();

brightness = 75;
FastLED.setBrightness(brightness);
duration = 10;
for (int i = 0; i < 100; i++)
    Random();

brightness = 175;
FastLED.setBrightness(brightness);
duration = 5;
for (int i = 0; i < 250; i++)
    Random();

brightness = 255;
FastLED.setBrightness(brightness);
duration = 1;
for (int i = 0; i < 555; i++)
    Random();

brightness = 175;
FastLED.setBrightness(brightness);
duration = 5;
Purples();

brightness = 255;
FastLED.setBrightness(brightness);
duration = 1;
```

```

Purples();
}

//eee
void IRTranslate() {
  if (irrecv.decode(&results)) {
    delay(100);
    irrecv.resume();

    //stores the most recent input to avoid repeats
    //0xFFFFFFFF is the code received when a repeat is consecutive button
    pressed
    if (results.value == 0xFFFFFFFF)
      results.value = previousResult;
    else
      previousResult = results.value;

    Serial.println("Current result = " + (results.value, HEX));
    Serial.println("Previous result = " + previousResult);
    switch (results.value) {
      //-----
      case 0xFFA25D: //turns on and off the LEDS
        Serial.println("CH- button has been pressed");
        Serial.println("CH- = power");
        if (currentState == HIGH) { //to turn off
          currentState == LOW;
          FastLED.setBrightness(0);
          while (currentState == LOW)
            delay(50);
        }
        else { //to turn on
          currentState == HIGH;
          FastLED.setBrightness(brightness);
        }
        break;
      //-----
      case 0xFF22DD: //decreases the speed of transitions
        Serial.println("PREV button has been pressed");
        Serial.println("PREV = -speed");
        if (duration >= 50)
          duration -= 50;
        break;
      //-----
      case 0xFF02FD: //increases the speed of transitions
        Serial.println("NEXT button has been pressed");
        Serial.println("NEXT = +speed");
        if (duration <= 2500)
          duration += 50;
        break;
    }
  }
}

```

```

//-----
-----
case 0xFFC23D: //pauses the code and allows the current colour to only
be displayed
    Serial.println("PAUSE button has been pressed");
    Serial.println("PAUSE/PLAY");
    if (paused == false) { //to pause
        paused = true;
        while (paused == true)
            delay (250);
    }
    else //to play/continue
        paused = false;
    break;
//-----

case 0xFFE01F: //decreases the brightness of LEDs
    Serial.println("VOL- button has been pressed");
    Serial.println("VOL- = -brightness");
    if (brightness >= 15) {
        brightness -= 15;
        FastLED.setBrightness(brightness);
    }
    break;
//-----

case 0xFFA857: //increases the brightness of LEDs
    Serial.println("VOL+ button has been pressed");
    Serial.println("VOL+ = +brightness");
    if (brightness <= 240) {
        brightness += 15;
        FastLED.setBrightness(brightness);
    }
    break;
//-----

case 0xFF906F: //swaps between all the different transitions
    Serial.println("EQ button has been pressed");
    Serial.println("EQ = transition");
    if (transition == 5)
        transition = 0;
    else
        transition += 1;

    switch (transition) {
        case 0:
            while (transition == 0)
                Default();
            break;
        case 1:
            while (transition == 1)
                Fade(crease);
            break;
        case 2:
            while (transition == 2)

```



```

        Spiral();
        break;
    case 3:
        while (transition == 3)
            Reload();
        break;
    case 4:
        while (transition == 4)
            Random();
        break;
    case 5:
        while (transition == 5)
            Purples();
        break;
    }
    break;
//-----

case 0xFF38C7: //displays only purple
    Serial.println("5 button has been pressed");
    Serial.println("5 = purple");
    SetAllLEDs(255, 0, 255);
    break;
//-----

case 0xFF42BD: //display icy white
    Serial.println("7 button has been pressed");
    Serial.println("7 = icy white");
    SetAllLEDs(212, 235, 255);
    break;
//-----

case 0xFF4AB5: //display only white
    Serial.println("8 button has been pressed");
    Serial.println("8 = white");
    SetAllLEDs(255, 255, 255);
    break;
//-----

case 0xFF52AD: //display warm white
    Serial.println("9 button has been pressed");
    Serial.println("9 = warm white");
    SetAllLEDs(255, 244, 229);
    break;
//-----

//      default:
//          //Serial.println("One of the following buttons have been
pressed.\n\tCH \n\tCH+ \n\t0 \n\t+100 \n\t+200 \n\t1 \n\t2 \n\t3 \n\t4
\n\t6");
//          //CH=FF629D, CH+=FFE21D, 0=FF6897, +100=FF9867,
+200=FFB04F, 1=FF30CF, 2=FF18E7, 3=FF7A85, 4=FF10EF, 6=FF5AA5
//          Serial.println("One of the following buttons have been
pressed; CH, CH+, 0, +100, +200, 1, 2, 3, 4, 6");
//          break;
}

```

```

    }
}

void SetAllLEDs (byte r, byte g, byte b) { //sets all LEDS to a single
colour using a for loop to go through all of them
    for (int i = 0; i < LED_NUM; i++)
        LEDs[i] = CRGB(r, g, b);
    FastLED.show();
    delay(duration);
}

void FadeBrightness(byte r, byte g, byte b) { //steadily increases and then
decreases brightness of all LEDs based on the current colour
    for (byte i = 0; i < brightness; i++) {
        FastLED.setBrightness(i);
        SetAllLEDs(r, g, b);
        FastLED.show();
        delay(duration);
    }
    for (byte i = brightness; i > 0; i--) {
        FastLED.setBrightness(i);
        SetAllLEDs(r, g, b);
        FastLED.show();
        delay(duration);
    }
}

void UpdateCurrentLED() {
    //keeps track of which LED currently display / updated with a new colour
    //and then resets it back to index zero when it gets to the highest index
    if (currentLED < (LED_NUM - 1))
        currentLED++;
    else
        currentLED = 0;
    delay(duration);
}

void FadePreviousLED(byte r, byte g, byte b) { //a way to keep track of past
LEDs with new colours

    for (byte i = 1; i <= 17; i++) {
        //since there is no setBrightness function for a single LED
        //reducing the amount of red/green/blue will make it appear to be dimmer
        r = r * i / 17;
        g = g * i / 17;
        b = b * i / 17;

        //keeps tracks of the previous LEDs for upto 17 behind of the currentLED
    }
}

```

```

    if (i > currentLED)
        previousLEDs = LED_NUM - i - currentLED;
    else
        previousLEDs = currentLED - i;
}
}

void Default() { //goes through all 16777216 possible colours with a smooth
transition without any significant change
    for (byte i = 0; i < 255; i++) //red to yellow
        SetAllLEDs(255, i, 0);

    for (byte i = 255; i > 0; i--) //yellow to green
        SetAllLEDs(i, 255, 0);

    for (byte i = 0; i < 255; i++) //green to cyan
        SetAllLEDs(0, 255, i);

    for (byte i = 255; i > 0; i--) //cyan to blue
        SetAllLEDs(0, i, 255);

    for (byte i = 0; i < 255; i++) //blue to purple
        SetAllLEDs(i, 0, 255);

    for (byte i = 255; i > 0; i--) //purple to red
        SetAllLEDs(255, 0, i);
}

void Fade(byte crease) { //basically default transition but it slowly becomes
brighter and until max desired brightness
//and then slowly fades until completely dark

    for (byte i = 0; i < 255; i += crease) //red to yellow
        FadeBrightness(255, i, 0);

    for (byte i = 255; i > 0; i -= crease) //yellow to green
        FadeBrightness(i, 255, 0);

    for (byte i = 0; i < 255; i += crease) //green to cyan
        FadeBrightness(0, 255, i);

    for (byte i = 255; i > 0; i -= crease) //cyan to blue
        FadeBrightness(0, i, 255);

    for (byte i = 0; i < 255; i += crease) //blue to purple
        FadeBrightness(i, 0, 255);

    for (byte i = 255; i > 0; i -= crease) //purple to red
        FadeBrightness(255, 0, i);
}

```

```

void Spiral() {
    //allows for a single LED to be a colour while going all around in a
    circle not stopping while also steadily changing colours

    for (byte i = 0; i < 255; i += crease) { //red to yellow
        FastLED.clear();
        LEDs[currentLED] = CRGB(255, i, 0);
        FastLED.show();
        UpdateCurrentLED();
        delay(duration);
    }
    for (byte i = 255; i > 0; i -= crease) { //yellow to green
        FastLED.clear();
        LEDs[currentLED] = CRGB(i, 255, 0);
        FastLED.show();
        UpdateCurrentLED();
        delay(duration);
    }

    for (byte i = 0; i < 255; i += crease) { //green to cyan
        FastLED.clear();
        LEDs[currentLED] = CRGB(0, 255, i);
        FastLED.show();
        UpdateCurrentLED();
        delay(duration);
    }

    for (byte i = 255; i > 0; i -= crease) { //cyan to blue
        FastLED.clear();
        LEDs[currentLED] = CRGB(0, i, 255);
        FastLED.show();
        UpdateCurrentLED();
        delay(duration);
    }

    for (byte i = 0; i < 255; i += crease) { //blue to purple
        FastLED.clear();
        LEDs[currentLED] = CRGB(i, 0, 255);
        FastLED.show();
        UpdateCurrentLED();
        delay(duration);
    }

    for (byte i = 255; i > 0; i -= crease) { //purple to red
        FastLED.clear();
        LEDs[currentLED] = CRGB(255, 0, i);
        FastLED.show();
        UpdateCurrentLED();
        delay(duration);
    }
}

void Reload() { //like spiral transition but it affects all LEDs
    //as new LEDs are updated with new colours, old LEDs are replaced

```

```

//all while steadily changing colours
for (byte i = 0; i < 255; i += crease) { //red to yellow
  LEDs[currentLED] = CRGB(255, i, 0);
  FadePreviousLED(255, i, 0);
  FastLED.show();
  UpdateCurrentLED();
  delay(duration);
}
for (byte i = 255; i > 0; i -= crease) { //yellow to green
  LEDs[currentLED] = CRGB(i, 255, 0);
  FadePreviousLED(i, 255, 0);
  FastLED.show();
  UpdateCurrentLED();
  delay(duration);
}

for (byte i = 0; i < 255; i += crease) { //green to cyan
  LEDs[currentLED] = CRGB(0, 255, i);
  FadePreviousLED(0, 255, i);
  FastLED.show();
  UpdateCurrentLED();
  delay(duration);
}

for (byte i = 255; i > 0; i -= crease) { //cyan to blue
  LEDs[currentLED] = CRGB(0, i, 255);
  FadePreviousLED(0, i, 255);
  FastLED.show();
  UpdateCurrentLED();
  delay(duration);
}

for (byte i = 0; i < 255; i += crease) { //blue to purple
  LEDs[currentLED] = CRGB(i, 0, 255);
  FadePreviousLED(i, 0, 255);
  FastLED.show();
  UpdateCurrentLED();
  delay(duration);
}

for (byte i = 255; i > 0; i -= crease) { //purple to red
  LEDs[currentLED] = CRGB(255, 0, i);
  FadePreviousLED(255, 0, i);
  FastLED.show();
  UpdateCurrentLED();
  delay(duration);
}
}

void Random(){ //basically is just reload but each LED that is updated with
a new colour, is randomised
  LEDs[currentLED] = CRGB(random(255), random(255), random(255));
  FastLED.show();
  UpdateCurrentLED();
}

```

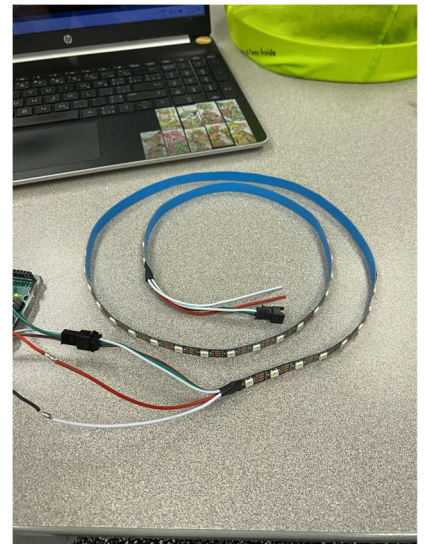
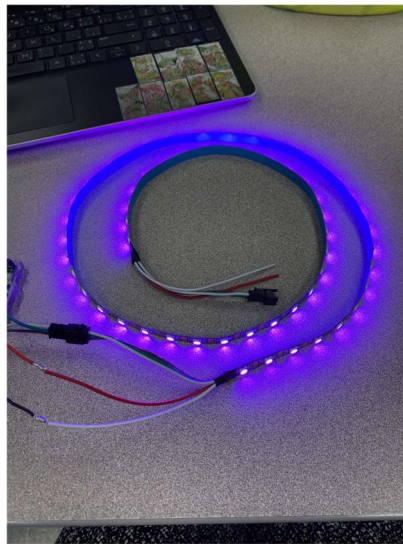
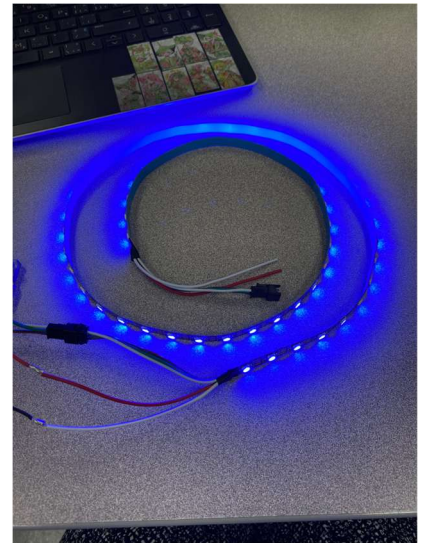
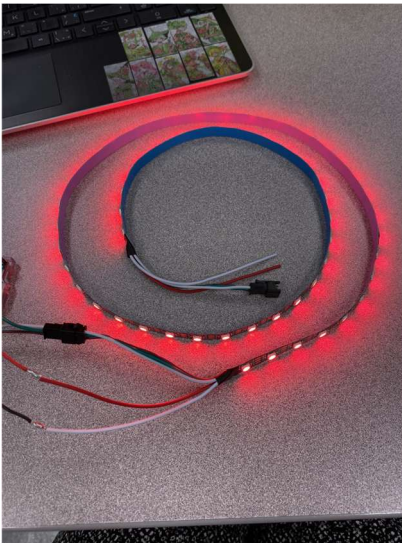
```
    delay(duration);  
}  
  
void Purples() { //basically just the default transition but it goes through  
all the possible purples  
    for (byte i = 0; i < 255; i++) // red to purple  
        SetAllLEDs(255, 0, i);  
  
    for (byte i = 255; i > 0; i--) //purple to blue  
        SetAllLEDs(i, 0, 255);  
  
    for (byte i = 0; i < 255; i++) //blue to purple  
        SetAllLEDs(i, 0, 255);  
  
    for (byte i = 0; i < 255; i++) //purple to white  
        SetAllLEDs(255, i, 255);  
  
    //reverse start  
    for (byte i = 255; i > 0; i--) //white to purple  
        SetAllLEDs(255, i, 255);  
  
    for (byte i = 255; i > 0; i--) //purple to blue  
        SetAllLEDs(i, 0, 255);  
  
    for (byte i = 0; i < 255; i++) //blue to purple  
        SetAllLEDs(i, 0, 255);  
  
    for (byte i = 255; i > 0; i--) //purple to red  
        SetAllLEDs(255, 0, i);  
}
```

Activity Log

Date (Jan)	Minutes Spent	Task Done
10	25	Strip and solder LED strip wires, since the wires were frayed
11	45	Test if all LEDs work along with each colour. (Specifically red, green, blue, white, and purple)
12	45	Figure out purple transitions
12	25	Receive hex codes for IR remote
13	70	Apply one way mirror film to glass
16	40	Measure and cut LED strip so it fits inside the rim
17	35	Resolder power, ground, data wires as a result of somehow snapping them off
18	60	Cut hole into back panel of clock
19	55	Glued LED strip to the inner rim of the back panel
20	40	Stuck mirror to back panel
23	15	Adjusted and taped LED strip wires to back panel
24	25	Clean smudges off of the mirrors
24	25	Stick velcro onto breadboard and power bank
10-24	N/A	Coded in between different tasks

Progress Pictures

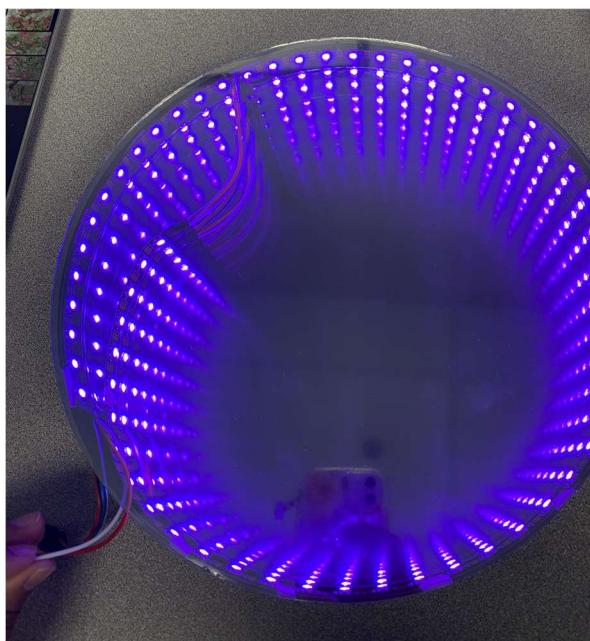
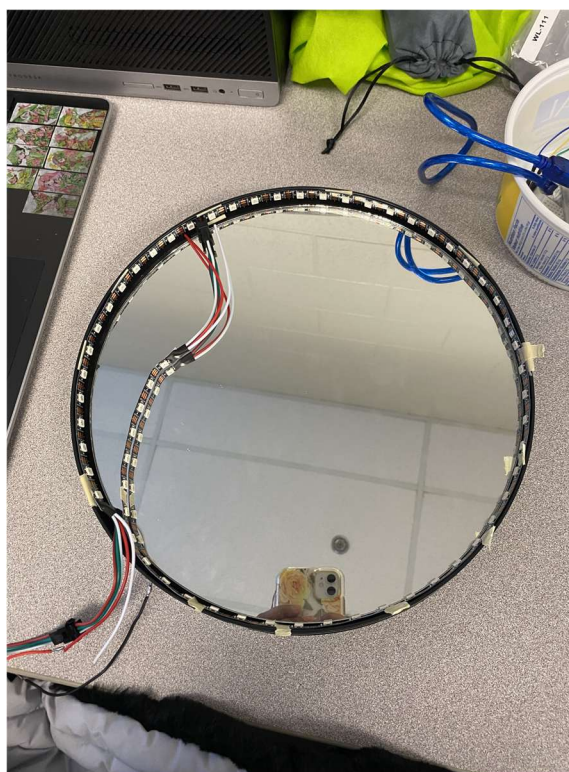
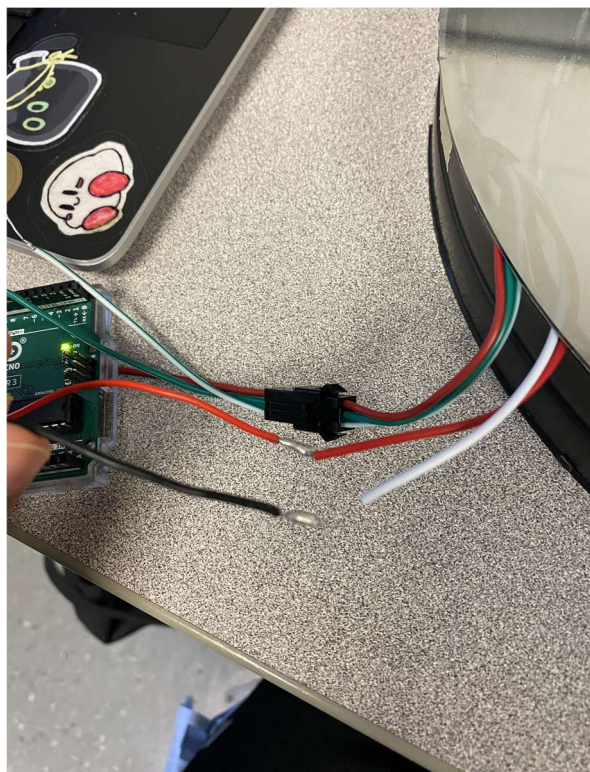
Testing out red, green, and blue to see each LED is capable of outputting each colour. Tested out white at max brightness, to test if each LED is capable of outputting at max power. Tested out purple since it is my favourite colour.



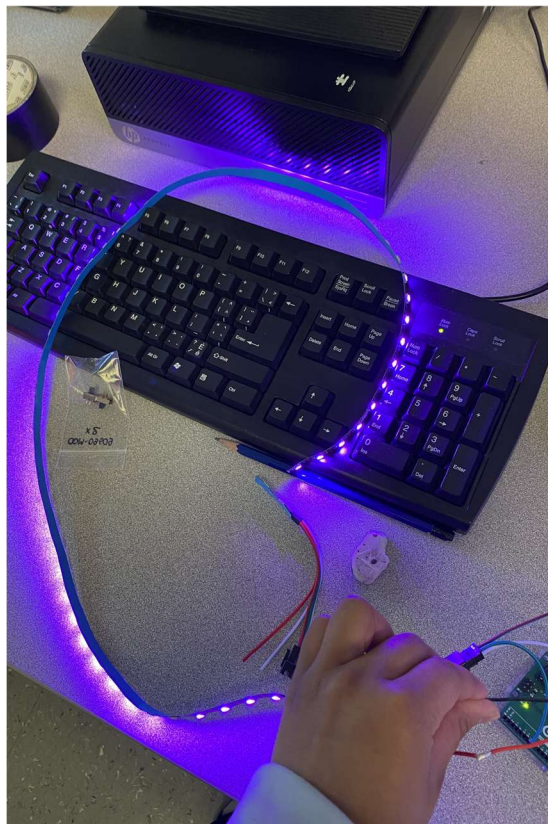
Applying the transparent one way mirror film to glass, then cutting off excess film, and finally cleaning the mirror.



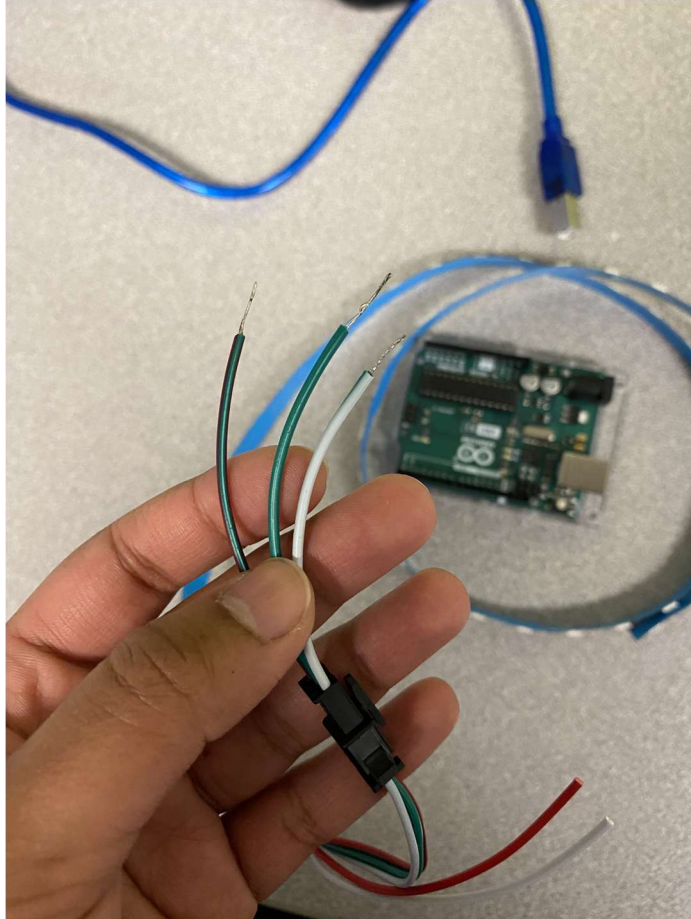
Temporarily placing the LED strip along the inner rim of the back panel. In the midst of doing so, I snapped the extra GND wire. Tested the one-way mirror to check for its reflectivity and clarity.



Measuring how much of the LED strip to cut off, then cutting it, and finally testing if it still works as it should be.



Recutting, restriping, and resoldering the LED strip's VCC, GND, and DATA wires, since they were not done as well as I wanted them to be. Also cutting off the extra VCC and GND wires since they were unnecessary in this case. They are used when there are multiple strips to add in additional power since they will not be as bright anymore.



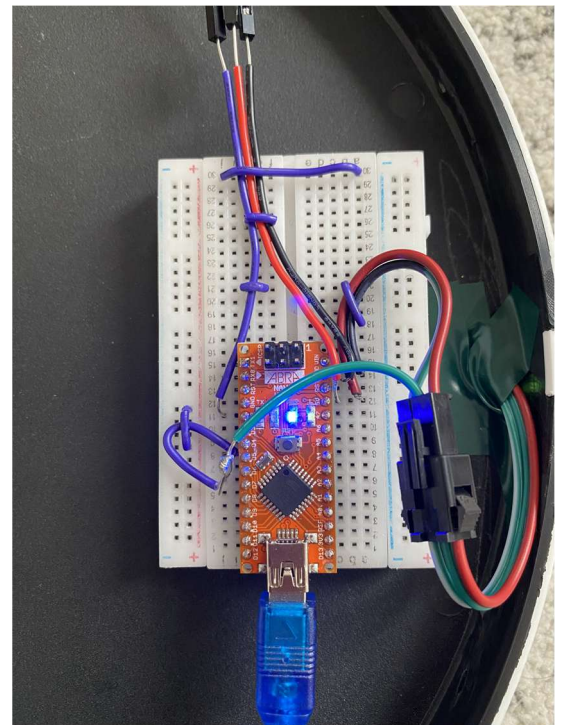
Sticking the LED strip along the inner rim of the back panel with gorilla glue. Then, using double sided tape to stick the mirror.



Making a hole in the back panel for the LED strip wires to come through, and then sealing the hole with tape. Finally, placing velcro for the IR remote, power bank, and breadboard.



Final Results:



Evaluation

A **strength** about my infinity mirror would be the smooth transitions between each colour. As each colour progresses, there is a very slight change in colour through the use of loops with an increment of one. These ever so small changes ensure that the transition does not look choppy.

Another **strength** would be the user interface. Through the use of an IR receiver and remote control, the user only has to press a designated button in order to switch between the various options. All products are heavily chosen and used for their convenience. If it is a hassle just to switch between options, would it even be used in the first place?

The first **weakness** would be the IR remote control buttons. Without looking at the operating instructions, it might be difficult to remember which button is for what. What I would have liked to do was get a few circular stickers and design a few icons for each button so each of their functions could be identified at first glance.

The second **weakness** would be the inner mirror sometimes coming off. However this was only an issue when it was placed the front facing the ground. It almost always goes back in place when slowly flipped back to the correct orientation. To resolve this, I would used a stronger form of adhesive to stick the mirror to the back panel. This time I used thick double sided tape with some type of squishy foam texture. Next time, I would either find stronger double sided tape or gorilla glue.

A **problem** that I faced was that the wires for the LED strip continuously frayed making it almost impossible to place into the solderless breadboard. In order to fix this, I simply applied a thin layer of solder to each of the frayed wires, ensuring they would solidify as one whole wire instead of a bunch of miniscule frayed ones.

Another **problem** that I encountered would be the IR receiver looking a bit out of place while it hung from the bottom of the frame. So I instead placed it at the top. At first I thought this would have been a problem with doing so, since it might have been hard for it to receive signals from all the way up there with the limited range. However, after receiving the component, I realised that the IR receiver itself can be tilted, which can counteract the issue with the range limit of the IR receiver.

In terms of any **changes**, if I were to make an infinity mirror again, I would like to use a better one-way mirror, in terms of both its clarity and reflectivity. This would allow for better practical use when it is turned off to function as a normal mirror, since as of now, it is just a tad bit blurry. Moreover, with a higher reflectiveness, there would be more illusions of the “infinite” LEDs, allowing for a deeper effect and appearance of the void of the abyss.

Self Assessment

	Level 4	Level 3	Level 2	Level 1
Communication	10 9 8	7	6	5
Title page Table of contents Layout Organization Pictures of progress Required Doc's Activity log Submitted digitally and in a well-organized folder. / 10	Exemplary layout and creative design throughout. No spelling / grammar errors. Easy to understand. Sections of the report are bolded/underlined and there is a clear division of topics / ideas.	Great layout and creative design Minor spelling / grammar errors. Sections of the report are mostly clear and there is a division of topics / ideas.	Good layout and design Some spelling / grammar errors that affect the overall flow of the report. Hard to understand at times. Sections of the report are organized but topics / ideas could be clearer.	Completed sections but design/ organization was not focused on Report is hard to understand and follow throughout. Missing 1-2 required report documents (deliverables) Missing hard copy OR digital copy. Only one is handed in.

10/10. Report is organised, free of any grammatical and spelling errors, and easy to navigate.

Application	Level 4	Level 3	Level 2	Level 1
	15 14 13 12	11 10.5	10 9	8 7.5
Construction and Technical skills in design/build. Construction Steps /15	Excellent workmanship and the project build is of excellent quality. Excellent creativity. All steps are listed in full detail and could easily be used by anyone to recreate your project step by step. All safety steps are included.	Project build is of good quality. Good overview of all steps taken but could not be easily be used by anyone to recreate your exact project	project build meets most requirements but is somewhat incomplete and/or not fully functioning. Minor issues with your order of operations – some information is omitted but still makes sense	project build is poor quality, incomplete and/or not fully functioning Major issues with the construction steps – out of order, missing key details that are pertinent to the overall execution of your project
	10 9 8	7	6	5
Programming Requirements met Comments (Header and internal documentation) / 10	Circuit works as per program. No repetitive code. Very efficient programming (eg. use of functions). Additional feature is added. Excellent level of commenting.	Meets requirements discussed in class. No repetitive code. Good level of commenting. Code is efficient.	Some requirements met or exceeded. Some repetitive code. Minimal level of commenting. Code is somewhat efficient.	Few requirements met or exceeded. Code is repetitive. No commenting. Code is not efficiently written.

15/15. Quality is of something that would be bought. While the steps are easy enough for anyone to understand, even without any prior knowledge.

10/10. All repetitive code is made into methods, and everything is commented thoroughly for easy comprehension.

Thinking	Level 4	Level 3	Level 2	Level 1
	5	4	3	2.5
Wire / Cable Management	Wires are creatively concealed, and the inner components are easily accessed for troubleshooting / repairs.	Wires are concealed and the inner components can be removed if needed.	Wires are not fully concealed throughout your model and the inner components	Wires are messy and hard to follow. Model cannot be easily accessed directly.
/5	Wiring follows all colour code conventions, minimal wire is used throughout and is neat throughout.	Minor issues with wiring: unnecessary long wires, colour code is not used.	would be removed if needed. Minor issues with wiring: unnecessary long wires, colour code is not used at all times	Minor issues with wiring: unnecessary long wires, colour code is not used at all times.
	5	4	3	2.5
Sketches Fritzing Diagrams /5	Excellent sketches (clearly labelled and proportional) show clear evidence of planning skills and provide at least one rough draft and one good copy that includes any changes made to the original design. Includes dimensions. Fritzing (Breadboard and Schematic) images are included, neat, components are easily identifiable, and accurate.	Neat sketches and Fritzing diagrams, correct placement of components in drawings but would be somewhat difficult to recreate entirely if given to a new person.	Minor components are incorrect or hard to locate and identify. It could be recreated only if an experienced person were to be given the task of re-building your final exam build.	Components are incorrect or hard to locate and identify. It would be hard for anyone to rebuild or create your project despite all relevant components being present. Lacks attention to detail.
	included like voltage requirements, pin layouts, full explanations, etc.	information (voltage requirements, I/O sequences) may not be explained in depth.		

5/5. All inner components have easy access for troubleshooting through the use of solderless clips, velcro, and a back panel which is easy and yet safe to take off. All wires follow colour convention and neatness.

5/5. Sketches are thorough and detailed while also showing development of the model. Fritzing diagrams are neat, accurate, identifiable, and thorough.

Knowledge	Level 4	Level 3	Level 2	Level 1
	10 9 8	7	6	5
Evaluation Report /10	Good evaluation. Complete description of strengths and weaknesses. Insightful and thoughtful changes are outlined with a complete list of problems faced.	Meets all requirements discussed in class but does not elaborate fully on what could be changed or how it could be improved	Missing 1-2 requirements.	Attempt is made but the connections made lack considerable depth and a detailed explanation is not given for strengths and weaknesses.
	5	4	3	2.5
Self-Evaluation Marked rubric Justification /5	Completed with a justification for each section.	All marks are calculated but there are some minor issues with your justifications	Marks are mostly complete and reasonable. Justifications are lacking in depth but there is effort displayed.	Marks are not reasonable and/or rationale is not justified.
	10 9 8	7	6	5
Circuit Summary Diagram Operating Instructions /10	Circuit explanations, instructions, and block diagram pictorial show an excellent level of understanding. This can be given to anyone and have it understood clearly whether they have a background in this field or not. No information is omitted. Key details are	Circuit explanations, instructions, and block diagram pictorial show a good level of understanding. Information is easy to follow but some of the more technical	1-2 requirements in this area are omitted or vague.	More than 2 key features are omitted which make your project's features not fully understandable. A lack of understanding is evident through the limited circuit explanations.

10/10. All weaknesses, strengths, and problems are outlined. For each problem there is a specific possible solution.

5/5. Each section has a reasonable justification.

10/10. Everything is easy enough for anyone to understand, even without any prior knowledge, while also maintaining an in-depth high-level understanding.

Total: 70/70.

Rubric

	Level 4	Level 3	Level 2	Level 1
Communication	10 9 8	7	6	5
Title page Table of contents Layout Organization Pictures of progress Required Doc's Activity log Submitted digitally and in a well-organized folder. / 10	Exemplary layout and creative design throughout. No spelling / grammar errors. Easy to understand. Sections of the report are bolded/underlined and there is a clear division of topics / ideas.	Great layout and creative design Minor spelling / grammar errors. Sections of the report are mostly clear and there is a division of topics / ideas.	Good layout and design Some spelling / grammar errors that affect the overall flow of the report. Hard to understand at times. Sections of the report are organized but topics / ideas could be clearer.	Completed sections but design/ organization was not focused on Report is hard to understand and follow throughout. Missing 1-2 required report documents (deliverables) Missing hard copy OR digital copy. Only one is handed in.
Application	Level 4	Level 3	Level 2	Level 1
	15 14 13 12	11 10.5	10 9	8 7.5
Construction and Technical skills in design/build. Construction Steps /15	Excellent workmanship and the project build is of excellent quality. Excellent creativity. All steps are listed in full detail and could easily be used by anyone to recreate your project step by step. All safety steps are included.	Project build is of good quality. Good overview of all steps taken but could not be easily be used by anyone to recreate your exact project	project build meets most requirements but is somewhat incomplete and/or not fully functioning. Minor issues with your order of operations – some information is omitted but still makes sense	project build is poor quality, incomplete and/or not fully functioning Major issues with the construction steps – out of order, missing key details that are pertinent to the overall execution of your project
	10 9 8	7	6	5
Programming Requirements met Comments (Header and internal documentation) / 10	Circuit works as per program. No repetitive code. Very efficient programming (eg. use of functions). Additional feature is added. Excellent level of commenting.	Meets requirements discussed in class. No repetitive code. Good level of commenting. Code is efficient.	Some requirements met or exceeded. Some repetitive code. Minimal level of commenting. Code is somewhat efficient.	Few requirements met or exceeded. Code is repetitive. No commenting. Code is not efficiently written.

Thinking	Level 4	Level 3	Level 2	Level 1
	5	4	3	2.5
Wire / Cable Management	Wires are creatively concealed, and the inner components are easily accessed for troubleshooting / repairs.	Wires are concealed and the inner components can be removed if needed.	Wires are not fully concealed throughout your model and the inner components	Wires are messy and hard to follow. Model cannot be easily accessed directly.
/5	Wiring follows all colour code conventions, minimal wire is used throughout and is neat throughout.	Minor issues with wiring: unnecessary long wires, colour code is not used.	would be removed if needed. Minor issues with wiring: unnecessary long wires, colour code is not used at all times	Minor issues with wiring: unnecessary long wires, colour code is not used at all times.
	5	4	3	2.5
Sketches Fritzing Diagrams /5	Excellent sketches (clearly labelled and proportional) show clear evidence of planning skills and provide at least one rough draft and one good copy that includes any changes made to the original design. Includes dimensions. Fritzing (Breadboard and Schematic) images are included, neat, components are easily identifiable, and accurate.	Neat sketches and Fritzing diagrams, correct placement of components in drawings but would be somewhat difficult to recreate entirely if given to a new person.	Minor components are incorrect or hard to locate and identify. It could be recreated only if an experienced person were to be given the task of re-building your final exam build.	Components are incorrect or hard to locate and identify. It would be hard for anyone to rebuild or create your project despite all relevant components being present. Lacks attention to detail.
	included like voltage requirements, pin layouts, full explanations, etc.	information (voltage requirements, I/O sequences) may not be explained in depth.		

Knowledge	Level 4	Level 3	Level 2	Level 1
	10 9 8	7	6	5
Evaluation Report /10	Good evaluation. Complete description of strengths and weaknesses. Insightful and thoughtful changes are outlined with a complete list of problems faced.	Meets all requirements discussed in class but does not elaborate fully on what could be changed or how it could be improved	Missing 1-2 requirements.	Attempt is made but the connections made lack considerable depth and a detailed explanation is not given for strengths and weaknesses.
	5	4	3	2.5
Self-Evaluation Marked rubric Justification /5	Completed with a justification for each section.	All marks are calculated but there are some minor issues with your justifications	Marks are mostly complete and reasonable. Justifications are lacking in depth but there is effort displayed.	Marks are not reasonable and/or rationale is not justified.
	10 9 8	7	6	5
Circuit Summary Diagram Operating Instructions /10	Circuit explanations, instructions, and block diagram pictorial show an excellent level of understanding. This can be given to anyone and have it understood clearly whether they have a background in this field or not. No information is omitted. Key details are	Circuit explanations, instructions, and block diagram pictorial show a good level of understanding. Information is easy to follow but some of the more technical	1-2 requirements in this area are omitted or vague.	More than 2 key features are omitted which make your project's features not fully understandable. A lack of understanding is evident through the limited circuit explanations.

Bibliography

Finio, Ben. “Arduino-Controlled RGB Led Infinity Mirror.” *Instructables*, Autodesk, 17 Sept. 2013, <https://www.instructables.com/Arduino-controlled-RGB-LED-Infinity-Mirror/>.

A self-directed project by BEn Finio that used an Arduino UNO to program an RGB LED strip using a potentiometer to control each the red, green, and blue values in order to creat an infinity mirror.

Pattabiraman, Krishna. “How to Set up an IR Remote and Receiver on an Arduino.” *Circuit Basics*, 2 Nov. 2021, <https://www.circuitbasics.com/arduino-ir-remote-receiver-tutorial/>.

A tutorial by Kirshna Pattabiraman on how to wire and set an IR remote and receiver for an Arduino and explaining the concepts behind them.

Window Films, Gila. “Gila RTK500 Complete Window Film Application Kit, 1-Pack.” *Gila RTK500 Complete Window Film Application Kit, 1-Pack*, Amazon, 7 Apr. 2018, https://www.amazon.ca/Gila-Window-Films-RTK500-Application/dp/B002YXPITY/ref=sr_1_4?crid=1BJ7U1HFA2086&keywords=mirror%2Bfilm%2Bapplication%2Bkit&qid=1670131878&refinements=p_72%3A11192170011&rnid=11192166011&sprefix=mirror%2Bfilm%2Bapplication%2Bkit%2Caps%2C138&sr=8-4.

A tutorial that is included as a video and step-by-step instructions in the description and pictures of the product’s site.

Workshop, DroneBot. “Using IR Remote Controls with the Arduino.” *Using IR Remote Controls with the Arduino*, YouTube, 5 Jan. 2018, <https://www.youtube.com/watch?v=8E3ltjnbV0c>.

A detailed runthroug of how to use IR remote controls with an Arduino by the creator of DroneBot Workshop. He goes over the following topics;

- An introduction
- How IR remotes work
- How to “see” infrared light
- The IR sensor
- Decoding IR remote codes
- Getting manufacturers codes
- Repurposing an IR remote
- Building a custom IR control